

Chat With Your Own Document (DocChat)

Author: **Muhammad Aqib**

Problem Statement.

Large language models like Claude have a fundamental constraint: they can only process a limited amount of text in a single request (the "context window"). A typical corporate document, legal contract, or research paper may span hundreds of pages, far beyond what any model can ingest in one call. Simply uploading a PDF to a chat interface is not a scalable solution.

Beyond the technical constraint, there is an accuracy problem. AI models trained on general internet data will attempt to answer questions about documents they have never seen, producing plausible-sounding but fabricated answers, a phenomenon called hallucination. For professional or academic use cases, this is unacceptable.

DocChat solves both problems: it uses RAG to retrieve only the most relevant passages from a document at query time, keeping prompts concise, and it instructs the AI to answer exclusively from those retrieved passages eliminating hallucination by design.

ARCHITECTURE: Frontend (index.html on S3 + CloudFront) → API Gateway → Lambda → Bedrock Knowledge Base + Claude 3 Sonnet → DynamoDB → Response with citations back to browser.

System Architecture

3.1 Architecture Overview

The system is structured as two independent pipelines that share underlying storage. This separation is intentional, ingestion and querying have very different performance characteristics and failure modes, and decoupling them makes each simpler and more reliable.

Pipeline 1 — Ingestion (event-driven, runs once per document): User uploads PDF → S3 stores it → Lambda triggers → Bedrock Knowledge Base automatically extracts text, chunks it, generates embeddings via Titan, and stores vectors in Aurora Serverless.

Pipeline 2 — Query (synchronous, runs on every user question): User types question → API Gateway → Lambda embeds question, retrieves top-5 relevant chunks from Knowledge Base, injects them into a Claude prompt with conversation history, calls Claude → answer with citations returned to browser.

3.2 AWS Services Used

Service	Role in Project	Why This Service
S3 (PDF bucket)	Stores uploaded PDFs	Infinitely scalable, durable object storage. Event notifications trigger ingestion automatically.
S3 (hosting bucket)	Serves the browser UI	Static website hosting — no web server needed. index.html becomes a globally accessible URL.
CloudFront	CDN + HTTPS for the UI	Provides free SSL certificate and edge caching. Without it, S3 is HTTP-only and browsers block cross-origin API calls.
API Gateway	HTTPS endpoint for Lambda	Converts browser HTTP requests to Lambda invocation events. Handles CORS, rate limiting, and routing.
Lambda (pdf-chat-query)	RAG query orchestration	Serverless — scales to zero, no idle cost. Runs the retrieve → prompt → generate → respond pipeline.
Lambda (pdf-presign-url)	Secure S3 upload URLs	Generates 5-minute pre-signed URLs so the browser uploads directly to S3 without exposing credentials.
Lambda (pdf-kb-auto-sync)	Triggers KB ingestion	Fires on S3 PUT events. Calls StartIngestionJob so new PDFs are indexed automatically within seconds.
Bedrock Knowledge Base	Managed RAG infrastructure	Replaces what would be ~500 lines of manual code: PDF parsing, chunking, embedding, vector storage, and k-NN search.
Bedrock — Claude 3 Sonnet	Answer generation	State-of-the-art LLM. Reads retrieved context and generates grounded, cited answers. Never uses training data.
Bedrock — Titan Embeddings	Text to vectors	Converts text to 1536-dimension vectors used for semantic similarity search. Called by Knowledge Base internally.
Aurora Serverless v2	Vector database	Managed by Bedrock Knowledge Base. Stores chunk vectors with pgvector extension for k-NN search.
DynamoDB	Chat session memory	Stores every message per session. Lambda loads history on each request so Claude maintains conversation context.
IAM	Permissions and security	Roles grant Lambda exactly the permissions it needs. Principle of least privilege throughout.

3.3 Why Bedrock Knowledge Base Changes Everything

In a traditional RAG implementation, the developer must write code for every step of the ingestion pipeline: a PDF parser, a text chunker, calls to an embedding model API, a vector database client, index creation and management, and similarity search logic.

Bedrock Knowledge Base replaces all of this with a single managed service. You point it at an S3 bucket, configure the embedding model and vector store, and AWS handles everything else. The developer only needs to call two API methods: `StartIngestionJob` to trigger indexing, and `Retrieve` to search. This reduced the development complexity of this project by roughly 70%.

Implementation (step by step)

4.1 Prerequisites

All steps in this project are performed through the AWS Management Console. No command-line tools are required. The following are needed before starting:

- **AWS Account** — an AWS account for development and testing
- **AWS Region** — all services must be deployed in **us-east-1** (N. Virginia) where Bedrock is fully available
- **Test PDF** — any PDF document to verify the system works end to end

STEP 1 IAM Roles · grant services permission to talk to each other

IAM (Identity and Access Management) is the AWS permissions system. Before creating any other resource, we create two roles: one that Lambda will use to call Bedrock and DynamoDB, and one that Bedrock Knowledge Base will use to read from S3.

WHY: Why IAM first? Every AWS service call requires explicit permission. If a Lambda function tries to call Bedrock without an IAM role granting that permission, the call fails with `AccessDeniedException` regardless of how the code is written. Creating roles first prevents this class of error entirely.

Role 1 — pdf-chat-lambda-role (for Lambda functions)

Console Path: IAM → Roles → Create role → AWS service → Lambda

1. **Trusted entity type:** AWS service — Lambda

2. **Attach policies directly:** AmazonBedrockFullAccess, AmazonS3FullAccess, AmazonDynamoDBFullAccess
3. **Role name:** pdf-chat-lambda-role

Role 2 — bedrock-kb-role (for Knowledge Base ingestion)

Console Path: IAM → Roles → Create role → AWS service → Bedrock

1. **Trusted entity type:** AWS service → Other → Amazon Bedrock
2. **Attach policies:** AmazonS3ReadOnlyAccess, AmazonBedrockFullAccess
3. **Role name:** bedrock-kb-role

STEP 2 S3 Buckets · storage for PDFs and the browser UI

Two separate S3 buckets are created with different access configurations. The PDF bucket is private, only Lambda and Bedrock can access it. The hosting bucket is public; browsers worldwide need to load the HTML file.

PDF Storage Bucket

Console Path: S3 → Create bucket

1. **Bucket name:** choose a globally unique name, e.g. my-pdf-chat-docs-2026
2. **Region:** us-east-1
3. **Block Public Access:** Go into this section; uncheck only the bottom two settings (bucket policy blocks) — keep the top two (ACL blocks) on
4. **CORS Configuration:** Permissions tab → CORS → paste the JSON below

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["PUT", "GET", "HEAD"],
    "AllowedOrigins": ["*"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

NOTE: The CORS configuration on the PDF bucket is critical. Without it the browser cannot PUT files to S3 even with a valid pre-signed URL — the request is blocked at the network level before it reaches S3.

UI Hosting Bucket

Console Path: S3 → Create bucket

1. **Bucket name:** pdf-chat-ui-hosting (or any unique name)
2. **Block Public Access:** uncheck all four boxes — this bucket must be publicly readable
3. **Static website hosting:** Properties tab → enable → index document: index.html
4. **Bucket policy:** Permissions → Bucket policy → paste public read policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::pdf-chat-ui-hosting/*"
  }]
}
```

STEP 3 Enable Bedrock Model Access · unlock Claude and Titan

(newer version already have this access, this is the procedure if in case it is not)

By default, no Bedrock foundation models are available in any AWS account. Access must be explicitly requested per region. This is a one-time step that accepts the terms of service for each model provider.

Console Path: Amazon Bedrock → Model access → Manage model access

1. **Enable:** Titan Embeddings G1 - Text (amazon.titan-embed-text-v1)
2. **Enable:** Claude 3 Sonnet (anthropic.claude-3-sonnet-20240229-v1:0)
3. **Click Save changes** Approval is instant for these models

NOTE: Model access is region-specific. Enabling in us-west-2 does not enable in us-east-1.

STEP 4 Bedrock Knowledge Base · the core of the RAG pipeline

The Knowledge Base is the most important resource in this project. It replaces the entire manual ingestion pipeline, PDF parsing, text extraction, chunking, embedding, vector storage, and similarity search — with a single managed service.

Console Path: Amazon Bedrock → Knowledge bases → Create knowledge base

Part A — Basic configuration

1. **Name:** pdf-chat-kb

2. **IAM permissions:** Use an existing service role → bedrock-kb-role

Part B — Data source

3. **Data source name:** pdf-s3-source
4. **S3 URI:** Browse S3 → select the PDF storage bucket (The pdf S3 bucket that we created)
5. **Chunking strategy:** Default chunking (~300 tokens per chunk with overlap)

Part C — Embeddings model

6. **Embeddings model:** Titan Embeddings G1 - Text
7. **Dimensions:** 1536 (matches Titan output)

Part D — Vector store

8. **Vector store:** Amazon Aurora (Quick create — fully managed by Bedrock)
9. **Click Create knowledge base** Wait 3-5 minutes for Aurora provisioning
10. **Save the Knowledge Base ID** shown on the details page — needed for Lambda code

STEP 5 Lambda Functions · the three serverless functions

Three Lambda functions handle different responsibilities. Each is a standalone Python function that runs only when triggered, scales automatically, and costs nothing when idle.

5A — pdf-presign-url

Generates a temporary, signed S3 URL that the browser uses to upload PDFs directly to S3. This approach keeps the S3 bucket private while allowing controlled uploads — the browser never needs AWS credentials.

Console Path: Lambda → Create function → Author from scratch → Python 3.12 → pdf-chat-lambda-role

```
import json, boto3, os
from datetime import datetime
from botocore.config import Config

BUCKET = os.environ['S3_BUCKET']
REGION = os.environ.get('AWS_REGION', 'us-east-1')
s3 = boto3.client('s3', region_name=REGION, config=Config(signature_version='s3v4'))

def lambda_handler(event, context):
    if event.get('httpMethod') == 'OPTIONS':
        return respond(200, {})

    raw_body = event.get('body') or '{}'
```

```

body = json.loads(raw_body) if isinstance(raw_body, str) else raw_body
file_name = body.get('fileName', '').strip()

if not file_name: return respond(400, {'error': 'fileName is required'})
if not file_name.lower().endswith('.pdf'): file_name += '.pdf'

# Prefix with pdfs/ so Knowledge Base scans the correct folder
ts = datetime.utcnow().strftime('%Y%m%d%H%M%S')
key = f'pdfs/{ts}_{file_name}'

upload_url = s3.generate_presigned_url('put_object',
    Params={'Bucket':BUCKET, 'Key':key, 'ContentType':'application/pdf'},
    ExpiresIn=300)

return respond(200, {'uploadUrl': upload_url, 's3Key': key})

def respond(code, body):
    return {'statusCode':code,
        'headers':{'Content-Type':'application/json','Access-Control-Allow-Origin':'*'},
        'body':json.dumps(body)}

```

Environment variable:

S3_BUCKET = PDF bucket name

5B — pdf-kb-auto-sync

Triggered automatically by S3 whenever a new PDF is uploaded. Calls the Bedrock Knowledge Base StartIngestionJob API to begin indexing the new document. Without this Lambda, documents would only be available after a manual sync.

```

import os, json, boto3

KNOWLEDGE_BASE_ID = os.environ['KNOWLEDGE_BASE_ID']
DATA_SOURCE_ID = os.environ['DATA_SOURCE_ID']
REGION = os.environ.get('BEDROCK_REGION', 'us-east-1')
bedrock_agent = boto3.client('bedrock-agent', region_name=REGION)

def lambda_handler(event, context):
    for record in event['Records']:
        key = record['s3']['object']['key']
        print(f'New PDF uploaded: {key} — starting KB sync')

    response = bedrock_agent.start_ingestion_job(
        knowledgeBaseId=KNOWLEDGE_BASE_ID,
        dataSourceId=DATA_SOURCE_ID,
    )
    job_id = response['ingestionJob']['ingestionJobId']
    print(f'Ingestion started — Job ID: {job_id}')
    return {'statusCode': 200, 'body': json.dumps({'jobId': job_id})}

```

Environment variables: Put all these as environment variables.

**KNOWLEDGE_BASE_ID,
DATA_SOURCE_ID,
BEDROCK_REGION**

S3 Trigger:

Go to__ Configuration → Triggers → S3 → your PDF bucket → Event: PUT → Suffix: .pdf

5C — pdf-chat-query

The core of the application. Called on every user question. Orchestrates the full RAG pipeline: embed the question, retrieve relevant chunks from the Knowledge Base filtered to the selected document, load conversation history from DynamoDB, build a grounded prompt, call Claude, save the turn, and return the answer with citations.

```
import json, os, boto3, uuid
from datetime import datetime

KNOWLEDGE_BASE_ID = os.environ['KNOWLEDGE_BASE_ID']
REGION = os.environ.get('BEDROCK_REGION', 'us-east-1')
CLAUDE_MODEL_ID = os.environ.get('CLAUDE_MODEL_ID',
    'anthropic.claude-3-sonnet-20240229-v1:0')
S3_BUCKET = os.environ.get('S3_BUCKET', '')
TOP_K, MAX_HISTORY = 5, 6

bedrock_agent = boto3.client('bedrock-agent-runtime', region_name=REGION)
bedrock_rt = boto3.client('bedrock-runtime', region_name=REGION)
dynamodb = boto3.resource('dynamodb', region_name=REGION)
history_table = dynamodb.Table('ChatHistory')

def retrieve_chunks(question, s3_key=None):
    config = {'vectorSearchConfiguration': {'numberOfResults': TOP_K}}
    if s3_key and S3_BUCKET:
        # Filter to the specific document the user selected
        config['vectorSearchConfiguration']['filter'] = {
            'equals': {'key': 'x-amz-bedrock-kb-source-uri',
                'value': f's3://{S3_BUCKET}/{s3_key}'}}
    }
    resp = bedrock_agent.retrieve(knowledgeBaseId=KNOWLEDGE_BASE_ID,
        retrievalQuery={'text': question},
        retrievalConfiguration=config)
    chunks = []
    for r in resp.get('retrievalResults', []):
        s3_loc = r.get('location', {}).get('s3Location', {})
        meta = r.get('metadata', {})
        chunks.append( {'text': r['content']['text'], 'score': r.get('score', 0),
            'source': s3_loc.get('uri', ''), 'page': meta.get(
                'x-amz-bedrock-kb-chunk-page-number', 'N/A')})
    # Fall back to full KB search if filter returns nothing
    if not chunks and s3_key: return retrieve_chunks(question)
    return chunks

def get_history(session_id):
    resp = history_table.query(
```



```

        KeyConditionExpression=boto3.dynamodb.conditions.Key('session_id').eq(session_id),
        ScanIndexForward=False, Limit=MAX_HISTORY)
    return sorted(resp.get('Items',[]), key=lambda x: x['timestamp'])

def save_messages(session_id, question, answer):
    ts = datetime.utcnow().isoformat()
    history_table.put_item(Item={'session_id':session_id,'timestamp':ts+'-u',
                                'role':'user','content':question})
    history_table.put_item(Item={'session_id':session_id,'timestamp':ts+'-a',
                                'role':'assistant','content':answer})

def build_messages(question, chunks, history, doc_name=None):
    context = '\n\n---\n\n'.join(
        f[{c["source"].split("/")[-1]} p.{c["page"]}]'\n{c["text"]}' for c in chunks)
    system = f""""You are a PDF assistant. Use ONLY the context below.
    Answer ONLY from context. If not found say so. Cite file and page.'
    fCONTEXT:\n{context}""""
    messages = [{ 'role':m['role'],'content':m['content']} for m in history[-MAX_HISTORY:]]
    messages.append({ 'role':'user','content':question})
    return system, messages

def invoke_claude(system, messages):
    resp = bedrock_rt.invoke_model(modelId=CLAUDE_MODEL_ID,
        contentType='application/json', accept='application/json',
        body=json.dumps({'anthropic_version':'bedrock-2023-05-31',
                        'max_tokens':1024,'system':system,'messages':messages}))
    return json.loads(resp["body"].read())['content'][0]['text']

def lambda_handler(event, context):
    body = json.loads(event.get('body','{}'))
    question = body.get('question','').strip()
    session_id = body.get('session_id') or str(uuid.uuid4())
    s3_key = body.get('s3Key','').strip()
    if not question: return _r(400,{'error':'question required'})
    chunks = retrieve_chunks(question, s3_key or None)
    history = get_history(session_id)
    sys_p, msgs = build_messages(question, chunks, history)
    answer = invoke_claude(sys_p, msgs)
    save_messages(session_id, question, answer)
    return _r(200,{'answer':answer,'session_id':session_id,
        'sources':[{ 'file':c['source'].split("/")[-1],'page':c['page']} for c in chunks]})

def _r(code, body):
    return {'statusCode':code,
        'headers':{'Content-Type':'application/json','Access-Control-Allow-Origin':'*'},
        'body':json.dumps(body)}

```

Environment variables: Put all these in environment variables.

KNOWLEDGE_BASE_ID,
S3_BUCKET,
BEDROCK_REGION,
CLAUDE_MODEL_ID

Timeout: 60 seconds

Memory: 512 MB

STEP 6 DynamoDB · persistent conversation memory

DynamoDB stores every message in every chat session. Lambda is stateless — each invocation is independent. Without DynamoDB, Claude would have no memory of what was discussed earlier in the same conversation. DynamoDB provides permanent, low-latency storage with no schema to define beyond the partition key and sort key.

Console Path: DynamoDB → Create table

1. **Table name:** ChatHistory
2. **Partition key:** session_id (String)
3. **Sort key:** timestamp (String)
4. **Billing mode:** PAY_PER_REQUEST — no provisioned capacity, pay only per query

WHY: Why PAY_PER_REQUEST? This billing mode means there is no minimum charge. When nobody is using the app, the cost is zero. For a project with variable or unpredictable traffic, this is almost always the right choice.

STEP 7 API Gateway · expose Lambda as HTTPS endpoints

API Gateway is the HTTP front door. The browser cannot call Lambda directly. API Gateway receives HTTP requests, converts them to Lambda invocation events, and returns the Lambda response as an HTTP response. It also handles CORS headers, which are mandatory for browser-to-API communication across different domains.

Console Path: API Gateway → Create API → REST API → Regional

1. **Create resource /chat:** Actions → Create Resource → chat → enable CORS
2. **Create POST on /chat:** Actions → Create Method → POST → Lambda Proxy → pdf-chat-query
3. **Create resource /upload-url:** Actions → Create Resource → upload-url → enable CORS
4. **Create POST on /upload-url:** Actions → Create Method → POST → Lambda Proxy → pdf-presign-url
5. **Enable CORS on both resources:** Select resource → Actions → Enable CORS → replace existing headers
6. **Deploy:** Actions → Deploy API → New Stage → prod
7. **Copy the Invoke URL:** shown after deployment — paste into the app Settings modal

NOTE: Lambda Proxy Integration must be checked on both methods. Without it, API Gateway strips the request body before passing it to Lambda — the Lambda receives an empty event and returns errors.

STEP 8 CloudFront · HTTPS and global CDN for the UI

S3 static website hosting only supports HTTP. Modern browsers enforce HTTPS for security — any page served over HTTP that makes requests to HTTPS APIs triggers a mixed-content error and blocks the requests. CloudFront adds HTTPS with a free AWS-managed SSL certificate and also distributes the HTML file from 400+ edge locations worldwide for fast load times.

Console Path: CloudFront → Create distribution

1. **Origin domain:** select the S3 website endpoint (not the bucket itself)
2. **Viewer protocol policy:** Redirect HTTP to HTTPS
3. **Default root object:** index.html
4. **Create distribution** takes 5-10 minutes to propagate globally
5. **Copy the distribution domain:** d1234abcd.cloudfront.net — this is your live app URL

STEP 9 Browser Frontend · the complete UI in one HTML file

The entire frontend is a single index.html file. It contains the HTML structure, all CSS styling, and all JavaScript logic. No framework, no build tools, no dependencies to install. It is uploaded to the S3 hosting bucket and served by CloudFront.

Key frontend capabilities

- **Document sidebar** — drag-and-drop upload zone, document list with sync status indicators, document selection
- **Chat interface** — message thread with user and AI bubbles, typing indicator, auto-expanding input textarea
- **Source citations** — every AI answer includes clickable tags showing filename and page number
- **Multi-turn memory** — session ID persists across messages so Claude maintains conversation context
- **Settings modal** — API Gateway URL and bucket name stored in localStorage, no hardcoded credentials
- **Dark theme** — CSS custom properties (variables) drive the entire color system, easily customizable

Upload flow in the frontend

```
// 1. Browser calls Lambda to get a temporary upload URL
const resp = await fetch(`${apiUrl}/upload-url`, {
  method: 'POST',
  body: JSON.stringify({ fileName: file.name, fileType: 'application/pdf' })
});
const { uploadUrl, s3Key } = await resp.json();

// 2. Browser PUTs the file directly to S3 — no credentials needed
```

```
await fetch(uploadUrl, { method: 'PUT', body: file,
  headers: { 'Content-Type': 'application/pdf' } });

// 3. Store the s3Key — sent with every question to scope search to this doc
activeDoc.s3Key = s3Key;
```

Query flow in the frontend

```
// Every question includes the s3Key of the selected document
// This lets Lambda filter Knowledge Base search to that PDF only
const body = {
  question: userQuestion,
  session_id: sessionId, // multi-turn memory
  s3Key: activeDoc.s3Key // document scoping
};
const data = await fetch(`${apiUrl}/chat`, {
  method: 'POST', body: JSON.stringify(body)
}).then(r => r.json());

// Render answer with source citations
appendMessage('ai', data.answer, data.sources);
// data.sources = [{ file: 'report.pdf', page: '12', score: 0.89 }]
```

To deploy: upload `index.html` to the hosting S3 bucket → set Content-Type: text/html → open the CloudFront URL in any browser → click ⓘ → enter your API Gateway URL and PDF bucket name → Save.

9. Results and Capabilities

The completed DocChat application demonstrates the following capabilities:

- **PDF Upload and Ingestion:** users can upload PDFs up to 50MB directly from the browser. Documents are automatically indexed within 30-60 seconds for typical PDFs.
- **Semantic Question Answering:** questions are answered using meaning-based search, not keywords. Synonyms, paraphrasing, and related concepts all find relevant passages.
- **Document Scoping:** each conversation is scoped to the selected document. Questions about document A do not return results from document B.
- **Source Citations:** every answer includes the source filename and page number. Users can verify every claim against the original document.
- **Multi-turn Conversation:** the system maintains context across questions in the same session. Follow-up questions like "tell me more about that" work correctly.
- **Zero Hallucination:** Claude is structurally prevented from using its training data. If an answer is not in the document, the system says so explicitly.
- **Serverless Scalability:** the architecture scales from zero to thousands of concurrent users with no manual intervention.

10. Conclusion

In this project every component from vector storage to LLM inference to CDN delivery is handled by AWS. The most significant architectural insight from this project is the value of Bedrock Knowledge Base as an abstraction. In a traditional implementation, the RAG ingestion pipeline requires a PDF parser, a text chunker, an embeddings API client, a vector database, index management logic, and similarity search code. Bedrock Knowledge Base eliminates all of this, reducing a 500-line multi-file system to a two-call API interaction.

The project also demonstrates important AWS patterns that apply broadly: pre-signed URLs for secure browser-to-S3 uploads, Lambda Proxy Integration for clean HTTP-to-Lambda bridging, DynamoDB for serverless session state, and IAM roles for service-to-service authorization. These patterns appear in virtually every modern AWS application.

Future enhancements that would make this production-ready: Amazon Cognito for user authentication and per-user document isolation; AWS WAF on API Gateway to prevent abuse; CloudWatch dashboards for monitoring; support for other document formats (Word, PowerPoint) using Amazon Textract; and a scheduled re-sync to handle document updates.

Appendix — Quick Reference

A. All AWS Resources Created

Resource	Name / Identifier	Purpose
IAM Role	pdf-chat-lambda-role	Grants Lambda access to Bedrock, S3, DynamoDB
IAM Role	bedrock-kb-role	Grants Bedrock KB access to S3
S3 Bucket	[your-pdf-bucket]	Stores uploaded PDFs in pdfs/ prefix
S3 Bucket	pdf-chat-ui-hosting	Hosts index.html as static website
CloudFront	[distribution-id]	HTTPS CDN for the UI
Bedrock KB	pdf-chat-kb	Managed RAG vector store
Bedrock Model	claude-3-sonnet	Answer generation
Bedrock Model	titan-embed-text-v1	Text embedding
Lambda	pdf-chat-query	RAG query pipeline
Lambda	pdf-presign-url	Secure upload URL generation
Lambda	pdf-kb-auto-sync	Auto-triggers KB ingestion on upload
API Gateway	pdf-chat-api	REST API with /chat and /upload-url

Resource	Name / Identifier	Purpose
DynamoDB Table	ChatHistory	Stores session conversation history

B. Environment Variables Reference

```
# pdf-chat-query Lambda
KNOWLEDGE_BASE_ID = <your-kb-id>
S3_BUCKET         = <your-pdf-bucket-name>
BEDROCK_REGION    = us-east-1
CLAUDE_MODEL_ID   = anthropic.claude-3-sonnet-20240229-v1:0

# pdf-presign-url Lambda
S3_BUCKET         = <your-pdf-bucket-name>
AWS_REGION        = us-east-1

# pdf-kb-auto-sync Lambda
KNOWLEDGE_BASE_ID = <your-kb-id>
DATA_SOURCE_ID    = <your-data-source-id>
BEDROCK_REGION    = us-east-1
```

C. Key Troubleshooting Reference

```
Error: AccessDeniedException
→ Lambda role is missing a permission. Check IAM → Roles → pdf-chat-lambda-role

Error: CORS blocked in browser
→ Enable CORS in API Gateway → redeploy → verify Lambda returns CORS headers

Error: 403 on S3 PUT
→ Check Block Public Access settings (bottom 2 must be OFF on PDF bucket)
→ Verify CORS config is saved on the PDF bucket

Error: fileName is required
→ Lambda Proxy Integration is not enabled on the API Gateway method

Documents not syncing
→ Check auto-sync Lambda trigger points to PDF bucket, not hosting bucket
→ Verify file is uploaded to pdfs/ prefix in the PDF bucket

Answers from wrong document
→ Verify s3Key is being sent from frontend in the request body
→ Check S3_BUCKET env variable matches the actual PDF bucket name
```